



第五章 分治法（一）

朱容波

Email: rongbozhu@126.com

Office: 一综B323

华中农业大学信息学院



本章内容

- 基本概念
 - 引言
 - Two simple examples (两个简单例子)
 - Selection sort (选择排序)
 - Insertion sort (插入排序)
- 类似数学归纳法的一些策略
 - Radix sort (基数排序)
 - Integer exponentiation (整数幂)
 - Evaluating polynomials (多项式求值)
 - Generating permutations (生成排列)
 - Finding the majority element (寻找多数元素)

华中农业大学信息学院



递归技术

- 递归最简单的描述是将一个问题分割成一个或多个的子问题，这些子问题与原问题具有相同的结构，然后组合这些子问题的解决方案，以获得原来的问题的解决方案的过程。



5.1 引言

- 数学方面

- 考虑一个带有参数 n 的问题，在问题的实例中 n 通常表示事物的数目。
- 当我们寻找这类问题的解时，有时候从求解一个带有小一点参数的相同问题开始，例如参数是 $n-1$, $n/2$ 等，然后再把解推广到包含所有的 n 个物体。
- 这样，问题的解决会比较容易些。这种方法基于众所周知的数学归纳法证明技术。



引言



递归思想

- 从本质上说, 给出一个带有参数 n 的问题, 用归纳法设计一个算法是基于这样一个事实, 如果我们知道如何求解带有参数小于 n 的同样问题(它被称为归纳假设), 那么我们的任务就化为如何把解法扩展到带有参数 n 的实例。
- 这种设计技术的一个优点是(一般地, 对于所有的递归算法来说都是如此), 它所设计出的算法的正确性证明已自然地嵌入了算法的描述中了。



例1 阶乘函数

- 阶乘函数可递归地定义为

$$n! = \begin{cases} 1 & n = 0 \\ n(n-1)! & n > 0 \end{cases}$$

边界条件

递归方程

- 边界条件与递归方程是递归函数的二个要素，递归函数只有具备了这两个要素，才能在有限次计算后得出结果。

华中农业大学信息学院



例2 Fibonacci数列

- 无穷数列1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ..., 被称为Fibonacci数列。它可以递归地定义为:

$$F(n) = \begin{cases} 1 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

边界条件

递归方程

华中农业大学信息学院



Fibonacci数列递归算法

第n个Fibonacci数可递归地计算如下:

```
int fibonacci(int n)
{
    if (n <= 1) return 1;
    return fibonacci(n-1)+fibonacci(n-2);
}
```

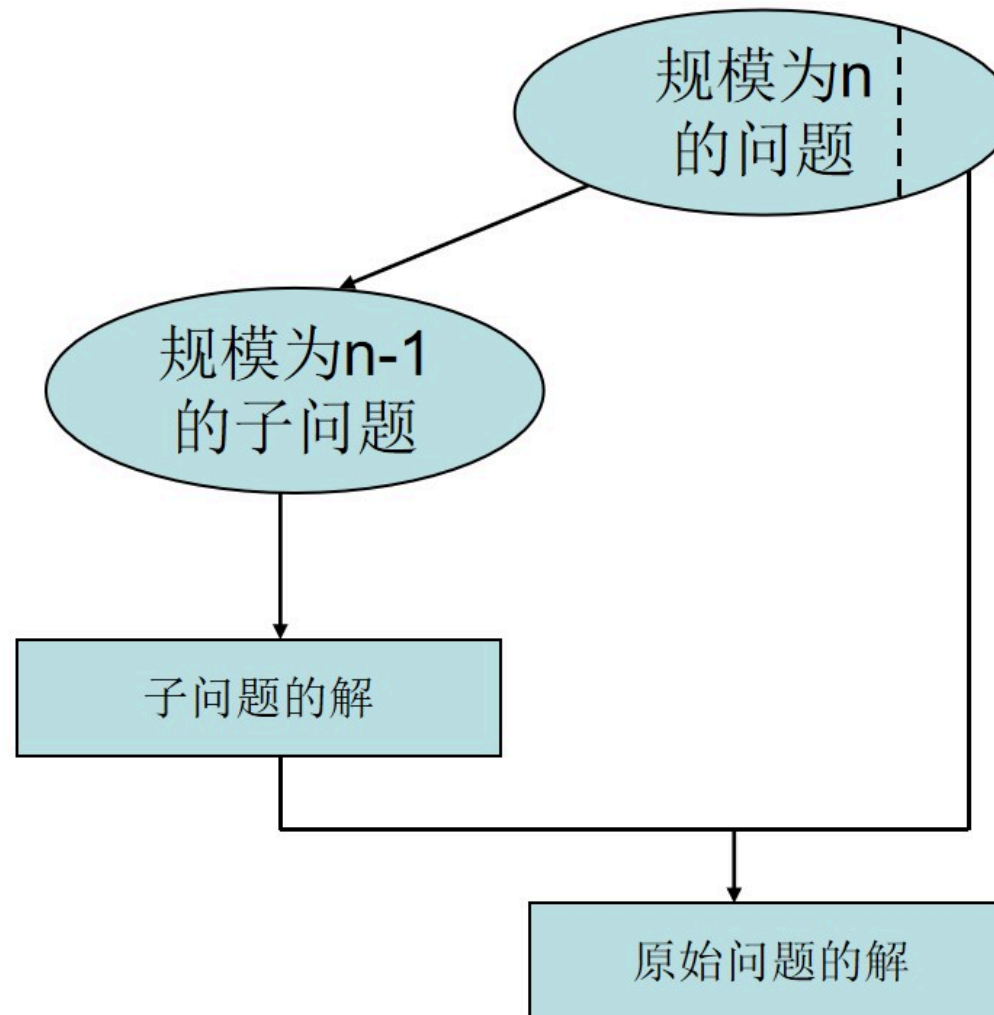


归纳法的基本思想

- 将规模为 n 的问题递减为规模为 $n-1$ 或 $n/2$ 的子问题,反复递减后对子问题分别求解,再建立子问题的解与原问题的解的关系。
- 归纳法有两个主要的变形:
 - 减常数(如1):每次迭代规模减1, 即 $n \rightarrow n-1$
 - 减因子(如 $1/2$):每次迭代规模减半, 即 $n \rightarrow n/2$



减（一）治技术



例如：

$$f(n) = a^n$$

$$f(n) = f(n-1) * a \quad n > 1$$

$$f(n) = a \quad n = 1$$

华中农业大学信息学院



5.2 两个简单的例子

- 先用两个简单的例子来展示归纳法技术的本质。
 - Selection sort (选择排序)
 - Insertion sort (插入排序)

华中农业大学信息学院



回顾 选择排序的迭代算法

📚 基本思想: 设 $A[1..n]$ 为包含 n 个元素的数组

- 首先, 我们找到数组中的最小元素, 将它存在 $A[1]$.
- 接着, 我们从剩下的 $n-1$ 个元素中找到最小的元素, 将它存在 $A[2]$.
- 重复以上过程, 知道整个数组中第二大的元素存在 $A[n-1]$, 则算法停止.

An example

- $A[1..11] = 6, 10, 9, 5, 3, 11, 4, 8, 1, 2, 7$

6	10	9	5	3	11	4	8	1	2	7
---	----	---	---	---	----	---	---	---	---	---

i

j

k

.....



Review Algorithm SELECTIONSORT (选择排序的迭代算法)

Input: 包含 n 个元素的数组 $A[1..n]$.

Output: $A[1..n]$ 中的元素非递减排列

1. **for** $i \leftarrow 1$ **to** $n-1$
2. $k \leftarrow i$
3. **for** $j \leftarrow i+1$ **to** n {找到第 i 个最小元素}
4. **if** $A[j] < A[k]$ **then** $k \leftarrow j$
5. **end for**
6. **if** $k \neq i$ **then** 交换 $A[i]$ and $A[k]$
7. **end for**

华中农业大学信息学院



5.2.1 Selection sort (选择排序)



采用归纳法的基本思想

- 令 $A[1..n]$ 作为算法的输入，由归纳法假设我们知道如何对后 $n-1$ 个元素排序，即对 $A[2..n]$ 排序。对某个 j ， $1 \leq j \leq n$ ，设 $A[j]$ 是最小值。
- 首先，如果 $j \neq 1$ ，我们交换 $A[1]$ 和 $A[j]$ ，即找到数组中的最小元素，将它存在 $A[1]$ 。
- 然后由归纳法假设，我们已知如何对 $A[2..n]$ 排序，因此可对在 $A[2..n]$ 中的元素递归的排序



算法 5.1 SELECTIONSORTREC

输入: n 个元素的数组 $A[1...n]$ 。

输出: 按非降序排列的数组 $A[1...n]$

1.sort(1)

过程 sort(i) {对 $A[i...n]$ 排序}

1. if $i < n$ then
2. $k \leftarrow i$
3. for $j \leftarrow i+1$ to n
4. if $A[j] < A[k]$ then $k \leftarrow j$ /* 永远使 k 指向 $i+1$ 到 n 中最小的那个数 */
5. end for
6. if $k \neq i$ then 互换 $A[i]$ and $A[k]$
7. sort($i+1$)
8. end if

华中农业大学信息学院



时间复杂度

- 令 $C(n)$ 表示元素比较的次数
 - 显然, $C(1) = 0$.
 - 过程sort第一次被调用时 $i = 1$, 数组中有 n 个元素, 因此在这次调用中, 元素比较的总次数由两部分元素比较次数之和组成, 即 $n-1$ 次元素比较次数加上对 $A[2..n]$ 排序比较次数, 后一比较次数是所有对 $\text{sort}(i+1)$ 的递归调用的总和。而从递归调用得到的元素比较次数是 $C(n-1)$ 。把这些放在一起, 有:

$$C(n) = \begin{cases} 0 & \text{if } n = 1 \\ C(n-1) + (n-1) & \text{if } n \geq 2 \end{cases}$$

这个递推式的解是

$$C(n) = \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2}$$

- 由于运行时间和元素的比较次数成线性关系, 所以说它是 $\Theta(n^2)$ 的。

选择排序的递归形式

SELECTIONSORTREC

输入: n 个元素的数组 $A[1..n]$ 。

输出: 按非降序排列的数组
 $A[1..n]$

1.sort(1)

过程 sort(i) {对 $A[i..n]$ 排序}

1. if $i < n$ then

2. $k \leftarrow i$

3. for $j \leftarrow i+1$ to n

4. if $A[j] < A[k]$ then $k \leftarrow j$

5. end for /* 永远使 k 指向
 $i+1$ 到 n 中最小的那个数 */

6. if $k \neq i$ then 交换 $A[i]$
and $A[k]$

7. sort($i+1$)

8. end if

选择排序的迭代形式

SELECTIONSORT Input: 包含 n 个元素的数组 $A[1..n]$ 。

Output: $A[1..n]$ 中的元素非递减排列

1. for $i \leftarrow 1$ to $n-1$

2. $k \leftarrow i$

3. for $j \leftarrow i+1$ to n {找到第
it 个最小元素}

4. if $A[j] < A[k]$ then $k \leftarrow j$

5. end for

6. if $k \neq i$ then 交换 $A[i]$
and $A[k]$

7. end for



5.2.2 Insertion sort (插入排序)

 插入排序的过程类似玩牌时整理手中纸牌的过程。它的基本思想是：每步将一个待排序的对象按照其关键字的大小，插到前面已经排好序的序列中的适当位置，直到全部对象插入完毕为止。



回顾 插入排序

迭代算法的基本思想：

- 从只包含一个元素的子数组 $A[1]$ 开始
- 接着, 将 元素 $A[2]$ 插入到子数组 $A[1]$ 的合适的位置, 这个位置可能在 $A[1]$ 之前或者 $A[1]$ 之后, 那么 $A[1..2]$ 变为有序的子数组
- 然后, 插入 $A[3]$, $A[4]$, 直到所有的元素都插入, 整个数组变为有序.

An example

- $A[1..11] = 6, 10, 9, 5, 3, 11, 4, 8, 1, 2, 7$

x =

10

6	10	9	5	3	11	4	8	1	2	7
---	----	---	---	---	----	---	---	---	---	---

i

j

.....



Review Algorithm INSERTIONSORT (插入排序的迭代形式版本)

Input: An array $A[1..n]$ of n elements.

Output: $A[1..n]$ sorted in nondecreasing order.

1. **for** $i \leftarrow 2$ **to** n
2. $x \leftarrow A[i]$ */**i指向即将插入的那个数**/*
3. $j \leftarrow i-1$
4. **while** $(j > 0)$ **and** $(A[j] > x)$
5. $A[j+1] \leftarrow A[j]$ */**向右移动所有比x大的元素**/*
6. $j \leftarrow j-1$
7. **end while**
8. $A[j+1] = x$
9. **end for**

华中农业大学信息学院



5.2.2 Insertion sort (插入排序)



基于归纳法的插入排序的基本思想

- 假设我们知道如何对前 $n-1$ 个元素, 也就是数组 $A[1..n-1]$ 排序。
- 那么在对 $A[1..n-1]$ 排序后, 我们只要把 $A[n]$ 插入它的适当位置, 假设是位置 j , $1 \leq j \leq n$ 。
- 这个插入可能导致 $A[j+1], A[j+2], \dots, A[n-1]$ 移到位置 $j+2, j+3, \dots, n$ 。



算法5.2 INSERTIONSORTREC

输入: n 个元素的数组 $A[1...n]$

输出: 按非降序排列的数组 $A[1...n]$

1. $\text{sort}(n)$

流程 $\text{sort}(i)$ $/*$ 对 $A[1...i]$ 排序 $*/$

1. **if** $i > 1$ **then**

2. $x \leftarrow A[i]$ $/*i$ 指向即将要插入的那个元素 $*/$

3. $\text{sort}(i-1)$ $/*$ 对 $A[1...i-1]$ 排序 $*/$

4. $j \leftarrow i-1$

5. **while** $j > 0$ **and** $A[j] > x$

6. $A[j+1] \leftarrow A[j]$

7. $j \leftarrow j-1$

8. **end while**

9. $A[j+1] \leftarrow x$

10. **end if**

华中农业大学信息学院



算法复杂性分析

- Clearly, $C(1) = 0$.
- The first time Procedure sort is invoked $i = n$, and hence there are n elements in the array.
- In the **best case** there are 1 element comparisons plus the number of element comparisons that result from the recursive call $\text{sort}(i-1)$ to sort the array $A[1..n-1]$. Hence, we have

$$C(n) = \begin{cases} 0 & \text{if } n = 1 \\ C(n-1) + 1 & \text{if } n \geq 2 \end{cases} \Rightarrow C(n) = n - 1$$

- In the **worst case** there are $n-1$ element comparisons plus the number of element comparisons that result from the recursive call $\text{sort}(i-1)$ to sort the array $A[1..n-1]$. Hence, we have

$$C(n) = \begin{cases} 0 & \text{if } n = 1 \\ C(n-1) + (n-1) & \text{if } n \geq 2 \end{cases} \Rightarrow C(n) = \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2}$$

算法5.2 插入排序的递归形式 INSERTIONSORTREC

输入: n 个元素的数组 $A[1..n]$

输出: 按非降序排列的数组 $A[1..n]$

1. $\text{sort}(n)$

流程 $\text{sort}(i)$ $\text{/*对 } A[1..i] \text{ 排序*/}$

1. **if** $i > 1$ **then**

2. $x \leftarrow A[i]$

3. $\text{sort}(i-1)$ $\text{/*对 } A[1..i-1] \text{ 排序*/}$

4. $j \leftarrow i-1$

5. **while** $j > 0$ **and** $A[j] > x$

6. $A[j+1] \leftarrow A[j]$

7. $j \leftarrow j-1$

8. **end while**

9. $A[j+1] \leftarrow x$

10. **end if**

插入排序的迭代形式 INSERTIONSORT

Input: An array $A[1..n]$ of n elements.

Output: $A[1..n]$ sorted in nondecreasing order.

1. **for** $i \leftarrow 2$ **to** n

2. $x \leftarrow A[i]$

3. $j \leftarrow i-1$

4. **while** $(j > 0)$ **and** $(A[j] > x)$

5. $A[j+1] \leftarrow A[j]$

6. $j \leftarrow j-1$

7. **end while**

8. $A[j+1] = x$

9. **end for**



5.3 Radix sort (基数排序)

? 问题描述

- 令 $L=\{a_1, a_2, \dots, a_n\}$ 是一张有 n 个数的表，其中每个数恰好有 k 位数字，就是说每个数具有 $d_k d_{k-1} \dots d_1$ 形式，这里 d_i 是0到9中的一个数字。
- 我们研究一种排序算法对 L 进行非降序排列，使算法在几乎所有的实际应用中都以线性时间运行。
- 在这个问题中，我们对整数的大小 k 施行归纳，而不是对数 n 这个对象来用归纳法。



Radix sort (基数排序)



基本思想:

- 首先根据 d_1 位, 把数分到10个表 $L_0, L_1, \dots, L_9$ 中, 这样, 那些 $d_1=0$ 的数分到表 L_0 中, 那些 $d_1=1$ 的数分到表 L_1 中, 等等。
- 接下来, 表被用 $L_0, L_1, \dots, L_9$ 的顺序接起来。
- 再后来, 把它们按照 d_2 分到10张表中, 按顺序接起来。
- 在把它们按 d_k 分到表中并按顺序接起来后, 所有的数就排好了序。

华中农业大学信息学院



惊奇的发现

- Given: $A = \{23, 32, 34, 45, 61, 78\}$

23 32 34 45 61 78

Sort A according to the **least** significant digit



惊奇的发现

- Given: $A = \{23, 32, 34, 45, 61, 78\}$

23 32 34 45 61 78

61 32 23 34 45 78

Sort A according to the **most** significant digit

Success!

Example

- Given: $A = \{7467, 1247, 3275, 6792, 9187, 9134, 4675, 1239\}$.

7467	6792	9134	9134	1239
1247	9134	1239	9187	1247
3275	3275	1247	1239	3275
6792	4675	7467	1247	4675
9187	7467	3275	3275	6792
9134	1247	4675	7467	7467
4675	9187	9187	4675	9134
1239	1239	6792	6792	9187

Success!



算法 5.3 RADIXSORT

输入: 一张有 n 个数的表 $L=\{a_1 \dots a_n\}$ 和 k 位数字。

输出: 按非降序排列的 L 。

1. **for** $j \leftarrow 1$ **to** k
2. 准备10个空表 $L_0, L_1, \dots, L_9$.
3. **while** L 非空
4. $a \leftarrow L$ 中的下一元素; 删除 L 中的 a
5. $i \leftarrow a$ 中的第 j 位数字; 将 a 加入表 L_i 中
6. **end while**
7. $L \leftarrow L_0$
8. **for** $i \leftarrow 1$ **to** 9
9. $L \leftarrow L, L_i$ {将表 L_i 加入表 L 中}
10. **end for**
11. **end for**
12. **return** L

华中农业大学信息学院



算法分析

- 根据某一位数字把数分到表中去的方法做了k遍，每一遍的代价是 $\Theta(n)$ 时间，这样算法的时间是 $\Theta(kn)$ 。如果k是常数，运行时间就是 $\Theta(n)$ 。
- 算法用了 $\Theta(n)$ 空间，因为需要10个表，这些表全部的大小是 $\Theta(n)$ 。



5.4多项式求值(Horner's规则)

? 问题描述：

- 假设有 $n+2$ 个实数 $a_0, a_1, \dots, a_n$ 和 x 的序列，要对如下多项式求值。

$$P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

- 直接的方法是对每一项分别求值。
- 这种方法是十分低效的，因为它需要 $n+n-1+\dots+1=n(n+1)/2$ 次乘法。



更快的方法(Horner's rule)

- 基本思想

首先观察

$$\begin{aligned} p_n(x) &= a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0 \\ &= (((((a_n x + a_{n-1})x + a_{n-2})x + a_{n-3}) \dots)x + a_1)x + a_0 \end{aligned}$$

这种求值的安排称为Homer规则。

导出:

$$p_{n-1}(x) = a_n x^{n-1} + a_{n-1} x^{n-2} + \dots + a_2 x + a_1$$

得到以下更有效的方法

$$p_n(x) = x p_{n-1}(x) + a_0, \quad p_j(x) = x p_{j-1}(x) + a_{n-j}$$

华中农业大学信息学院



算法5.6 HORNER

Input: $n+2$ ~~$P_n(x)$~~ $P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$ ~~实数~~ $a_n, a_{n-1}, \dots, a_1, a_0$ ~~的序列~~

Output:

1. $p \leftarrow a_n$
2. **for** $j \leftarrow 1$ **to** n
3. $p \leftarrow xp + a_{n-j}$
4. **end for**
5. **return** p



算法分析

- 算法HORNER的复杂度代价是 n 次乘法和 n 次加法,
- 对比 $1=n(n+1)/2$ 次乘法
- 这是利用归纳假设的优点得到的令人瞩目的成绩。



5.5 拓扑排序

- 在大学学习的过程中，各门课程的学习是有先后顺序的，有些课程需要先修课程，有些则不需要；有些课程之间有先后的关系，有些课程则可以并行的进行。问题要求确定一个学习方案使得各门课程的学习能够有序进行。
- 拓扑排序问题：

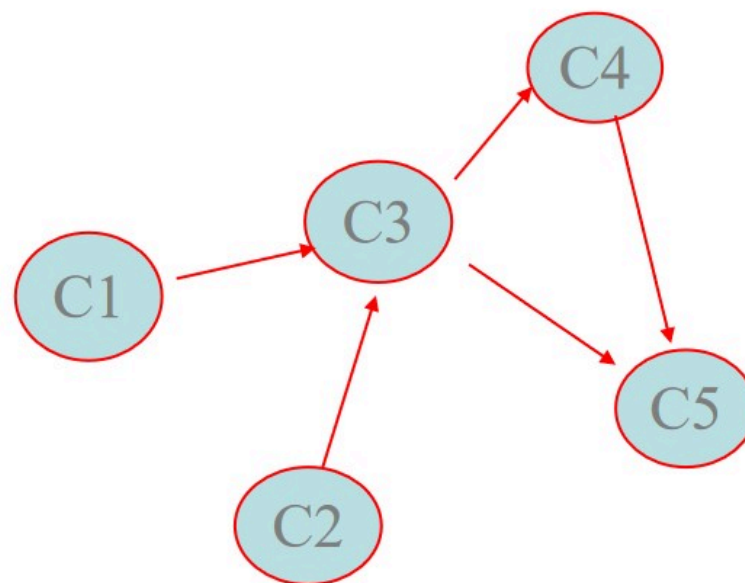
对给定的无环有向图，要求按照某种顺序列出它的顶点序列，使图的每一条边的起点总在结束顶点之前。

求拓扑序列的方法

- 方法: 减治法, 每次选择没有输入的点。

正常步骤:

- (1) 从DAG中找到一个没有前驱的节点输出
- (2) 删除以这个点为起点的边
- (3) 重复 (1) (2), 直到最后一个节点被输出



求得的解是: C1, C2, C3, C4, C5



5.6 生成组合对象的算法（生成排列问题）

- 问题描述
 - 研究对于数 $1, 2 \dots n$ 生成所有的排列问题，基于一个假设：我们可以生成 $n-1$ 个数的所有排列。



5.6 Generating permutations (生成排列)

- 要构造一个简单生成集合 $\{1,2,3,\dots,n\}$ 的所有排列的算法。观察发现：
 - 如果整数 n 从 $\{1,2,3,\dots,n\}$ 的一个排列中删除，那么结果则是 $\{1,2,3,\dots,n-1\}$ 的一个排列。

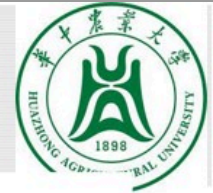


排列生成算法的设计

- 基本思想

假定可以生成 $n-1$ 个数的所有排列

- 生成数 $2, 3, \dots, n$ 的所有排列，并且在每个排列的前面加上数1
- 接下来，生成数 $1, 3, 4, \dots, n$ 的所有排列，并且在每个排列的前面加上数2。
- 重复这个过程直到最后生成 $1, 2, \dots, n-1$ 的所有排列，并且在每个排列的前面加上数 n 。



算法5.6 PERMUTATIONS1

Input: 正整数 n .

Output: 数 $1, 2, \dots, n$ 的所有可能排列

1. **for** $j \leftarrow 1$ **to** n //生成数组
2. $P[j] \leftarrow j$
3. **end for**
4. perm1(1)

过程 perm1(m)

1. **if** $m=n$ **then output** $P[1..n]$
2. **else**
3. **for** $j \leftarrow m$ **to** n
4. 互换 $P[j]$ and $P[m]$
5. perm1($m+1$)
6. 互换 $P[j]$ and $P[m]$
7. **comment:** at this point $P[m..n]=m, m+1, \dots, n$
8. **end for**
9. **end if**

华中农业大学信息学院



算法分析

- 由于存在有 $n!$ 个排列，过程perml的第1步共有 $nn!$ 次来
- 输出所有的排列。
- 现在计算for循环的循环次数。
 - 在第一次调用过程perml时， $m = 1$ ，因此for循环执行 n 次加上递归调用perml(2)的执行次数。
 - 由于当 $n=1$ 时反复次数是0，因此反复次数 $f(n)$ 能够用以下递推式表示

$$f(n) = \begin{cases} 0 & \text{if } n=1 \\ nf(n-1)+n & \text{if } n \geq 2 \end{cases}$$

- (书p105)



算法分析（继续）

令 $n!h(n)=f(n)$ (注意 $h(1)=0$). (书本p105)

那么 $n!h(n) = n(n-1)!h(n-1) + n$

这个递推式的解是

$$h(n) = h(1) + \sum_{j=2}^n \frac{1}{(j-1)!} = \sum_{j=1}^{n-1} \frac{1}{j!} < \sum_{j=1}^{\infty} \frac{1}{j!} = (e-1)$$

这里 $e=2.7182818\dots$ 因此,

$$f(n) = n!h(n) = n! \sum_{j=1}^{n-1} \frac{1}{j!} < 2n!$$

- 所以，算法的运行时间由输出语句来决定，也就是 $\Theta(nn!)$ 。



5.7 生成子集问题

问题描述

集合 A 的“幂集”是指以集合 A 的所有子集为元素组成的集合。

降低规模的减治法可以用来求幂集。

减治法在求解规模为 n 的问题时，需要得到规模为 $n-1$ 的问题的解。



减治法生成幂集

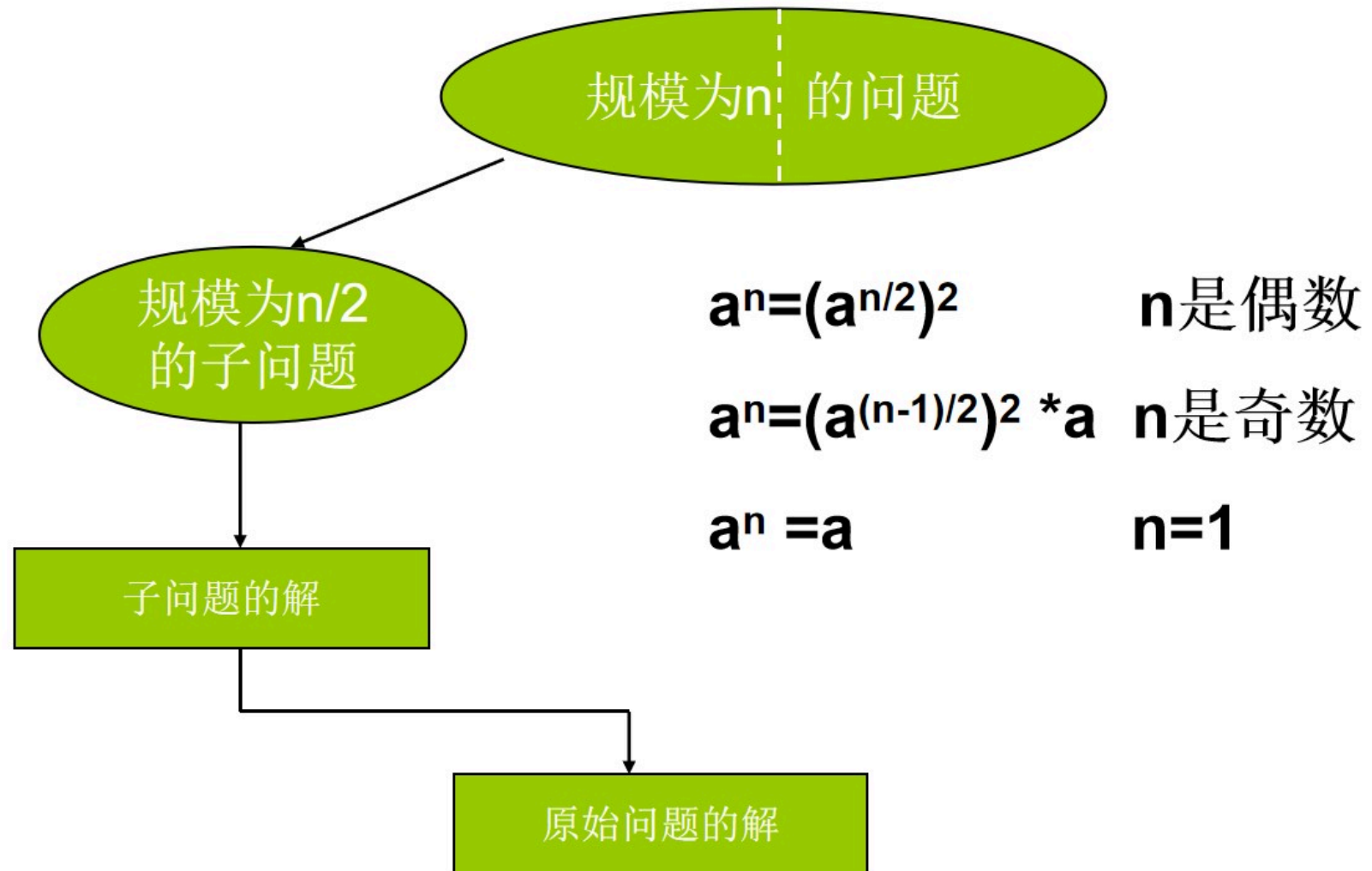
- 例 $n=3$

方法：在 $n=2$ 的幂集中加入元素3，在 $n=1$ 的幂集中加入元素2，在 $n=0$ 的幂集中加入元素1

- ϕ
- $\phi, \{1\}$ // $n=1$
- $\phi, \{1\}, \{2\}, \{1,2\}$ // 加入元素2
- $\phi, \{1\}, \{2\}, \{1,2\}, \{3\}, \{1,3\}, \{2,3\}, \{1,2,3\}$
// 加入元素3



减（半）治技术



华中农业大学信息学院



5.8 减常因子法（求整数幂问题）

? 问题描述

- 对求实数 x 的 n 次幂设计一个有效的算法
- 直接的方法
 - 对 x 用迭代方法自乘 n 次，这种方法是十分低效的，因为它需要 $\Theta(n)$ 乘法。按输入的大小来说，它是指数的(见1.14节)

? Can we find much more efficient method?



一种有效的方法

基本思路

- 令 $m = \lfloor n/2 \rfloor$, 假设已经知道如何计算 x^m , 那么有两种情况:

- 如果 n 是偶数, 那么

$$x^n = (x^m)^2$$

- 否则,

$$x^n = x(x^m)^2$$



算法5.4 EXPREC

Input: 实数 x 和非负整数 n 。

Output: x^n .

1. power(x , n)

流程 power(x , m) {计算 x^m }

1. **if** $m=0$ **then** $y \leftarrow 1$

2. **else**

3. $y \leftarrow \text{power}(x, \lfloor m/2 \rfloor)$

4. $y \leftarrow y^2$

5. **if** m 是奇数 **then** $y \leftarrow xy$

6. **end if**

7. **return** y



算法分析

- 假定每次乘法的时间是常数，那么这归纳法版本的算法的运行时间是 $\Theta(\log n)$ ，它对于输入的大小来说是线性的。



5.9 减常因子法（假币问题）

- 问题描述

有 n 个金币，其中一个是假币。这个假币的重量比真币的重量要轻一点，所有 $n-1$ 个金币的重量是一样的。现在你有一架天平，设计高效的算法（用最少的使用天平次数）找出那个假的金币。



假币问题解法

- 1、用减治法（减半）

把 n 个硬币分为两堆，每堆 $\lfloor n/2 \rfloor$ 个，每次称两堆。

易见 $W(1)=0$

$$W(n)=W(\lfloor n/2 \rfloor)+1$$

解得 $W(n)=\lfloor \log_2 n \rfloor$



假币问题解法

- 2、用减治法(减 $n/3$)

把 n 个硬币分为三堆，每堆 $\lfloor n/3 \rfloor$ 个，每次称任意二堆。

易见

$$W(1)=0$$

当 $n=1$

$$W(n)=W(\lfloor n/3 \rfloor)+1$$

当 $n>1$

解得 $W(n)=O(\lfloor \log_3 n \rfloor)$ ，结果比减半法更好。



假币问题实例： $n=9$

- 用方法1（减半法）需要称 3 次。
- 用方法2（减 $n/3$ 法）需要称 2 次。

过程：

将9个金币分为3个组，每组3个金币。

将其中的两组放在天平的两边，如果有一边轻，说明假的金币就在这个组里。如果两边一样重，说明假的金币在第三个组里。

在有假币的组中，拿出两个金币，放在天平的两边，如果有一个轻，则这个轻的就是假币，如果两个一样重，则剩下的一个就是假币。